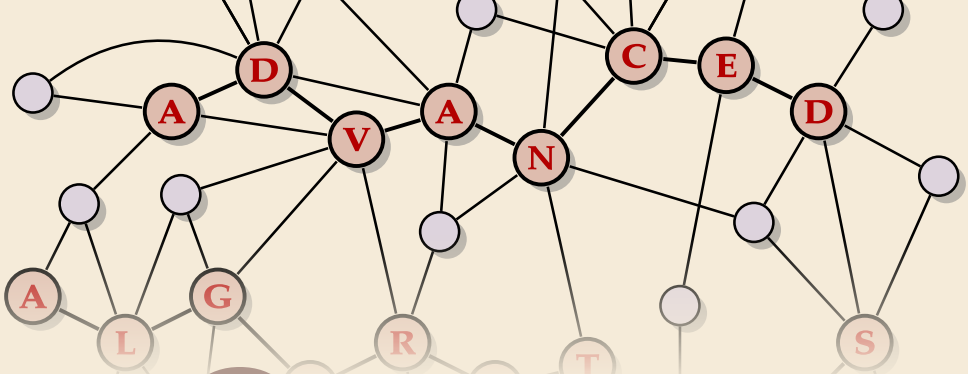


2 Complexity Theory Recap

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version `2026-10-15 13:22`

Outline

2 Complexity Theory Recap

2.1 P and NP Informally
2.2 Models of Computation
2.3 Turing Machines
2.4 The Classes P and NP
2.5 Nondeterminism = Verification
2.6 Karp-Reductions and NP-Completeness
2.7 Example of an NP-completeness proof
2.8 Important NP-Complete Problems
2.9 Optimization Problems

2.1 P and NP Informally

Hard problems

- Some algorithmic problems are **"hard nuts" to crack**.
  - e. g., the *Traveling Salesperson Problem (TSP):*

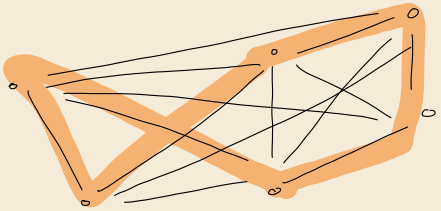
    Given: n cities $S_1, \ldots, S_n$,
    all $n(n-1)$ pairwise
    distances $d(S_i, S_j) \in \mathbb{N} \quad (i \neq j)$

    Goal: Shortest round trip through all cities

always exact, always correct — polytime

  - no general, efficient algorithm known!
    (despite decades of intensive research . . .)





$(n-1)!$ different tours

length of tour
= sum of connection distances

Hard problems



- Some algorithmic problems are **"hard nuts" to crack**.
  - e. g., the *Traveling Salesperson Problem (TSP):*

    Given: n cities $S_1, \ldots, S_n$,
    all $n(n-1)$ pairwise
    distances $d(S_i, S_j) \in \mathbb{N} \quad (i \neq j)$

    Goal: Shortest round trip through all cities

    always exact, always correct — polytime

  - no general, efficient algorithm known!
    (despite decades of intensive research . . .)

  ⇝ It *seems* as if there is no efficient algorithm for TSP!

Hard problems



- Some algorithmic problems are **"hard nuts" to crack**.
  - e. g., the *Traveling Salesperson Problem (TSP):*

    Given: n cities $S_1, \ldots, S_n$,
    all $n(n-1)$ pairwise
    distances $d(S_i, S_j) \in \mathbb{N} \quad (i \neq j)$

    Goal: Shortest round trip through all cities

  always exact, always correct — polytime

  - no general, efficient algorithm known!
    (despite decades of intensive research . . .)

  ⇝ It *seems* as if there is no efficient algorithm for TSP!

  - **But:** can we *prove* that?



  - Despite similarly intensive research: **No!** (not yet)

Doesn't sound like a shining example for theoretical computer science? . . . stay tuned!

United in incapacity



"I can't find an efficient algorithm, but neither can all these famous people."

Garey, Johnson 1979

Complexity Theory

- *Complexity theory* allows us to *compare* the *hardness* of algorithmic problems.



A: old problem
Consensus: hard



B: new problem
Status: unknown
(seems hard to *us* . . .)

Complexity Theory

- *Complexity theory* allows us to *compare* the *hardness* of algorithmic problems.



A: old problem
Consensus: hard

$\leq_p$



B: new problem
Status: unknown
(seems hard to *us* . . .)

Intuitive idea:

1. If A is a known hard nut, and
2. B is at least as hard as A,

then B is a hard nut, too!

Complexity Theory

- *Complexity theory* allows us to *compare* the *hardness* of algorithmic problems.



A: old problem
Consensus: hard

$\leq_p$



B: new problem
Status: unknown
(seems hard to *us* . . .)

Intuitive idea:

1. If A is a known hard nut, and
2. B is at least as hard as A,

then B is a hard nut, too!

Formally:

1. A is NP-hard: probably $\nexists$ eff. alg. for A
2. $A \leq_p B$: $\exists$ eff. alg. for $B \implies \exists$ eff. alg. for A

$\rightsquigarrow$ B is NP-hard: probably $\nexists$ eff. alg. for B!

P and NP – Intuitive Synopsis

$$P = NP?$$

P and NP – Intuitive Synopsis

- P = class of problems for which there is an algorithm A and a polynomial p such that A **solves** every instance I in time $O(p(|I|))$.
  - P for "polynomial" – i. e., all problems where a solution can be *found* by a (deterministic) algorithm in polynomial time.

$$P = NP?$$

P and NP – Intuitive Synopsis

- P = class of problems for which there is an algorithm A and a polynomial p such that A **solves** every instance I in time $O(p(|I|))$.
  - P for "polynomial" – i. e., all problems where a solution can be *found* by a (deterministic) algorithm in polynomial time.

- NP = class of problems for which there is an algorithm A and a polynomial p such that A can **verify** a given candidate solution $l(I)$ of a given instance I in time $O(p(|I|))$, i. e., check whether $l(I)$ solves I or not.
  - NP for "nondeterministically polynomial" – i. e., all problems where a solution can be *found* by a *nondeterministic* algorithm in polynomial time.
  - This is equivalent to the above characterization via verification.

P = NP?

P and NP – Intuitive Synopsis

- P = class of problems for which there is an algorithm A and a polynomial p such that A **solves** every instance I in time $O(p(|I|))$.
  - P for "polynomial" – i. e., all problems where a solution can be *found* by a (deterministic) algorithm in polynomial time.

- NP = class of problems for which there is an algorithm A and a polynomial p such that A can **verify** a given candidate solution $l(I)$ of a given instance I in time $O(p(|I|))$, i. e., check whether $l(I)$ solves I or not.
  - NP for "nondeterministically polynomial" – i. e., all problems where a solution can be *found* by a *nondeterministic* algorithm in polynomial time.
  - This is equivalent to the above characterization via verification.

- We know $\mathsf{P} \subseteq \mathsf{NP}$. We *think* $\mathsf{P} \subsetneq \mathsf{NP}$, i. e., $\mathsf{P} \neq \mathsf{NP}$.
  The question "P = NP?" is one of the famous **millenium problems** and arguably the **most important open problem of theoretical computer science**.

Clicker Question



Consider the problem CLIQUE-10: Given a graph G, does G contain a clique of size 10?
Which of the following are true?

- **A** CLIQUE-10 $\in$ P
- **B** CLIQUE-10 $\in$ NP
- **C** CLIQUE-10 is NP-complete
- **D** CLIQUE-10 $\notin$ NP
- **E** No idea.



→ *sli.do/advalg*

Clicker Question



Consider the problem CLIQUE-10: Given a graph G, does G contain a clique of size 10?

Which of the following are true?



- **A** CLIQUE-10 $\in$ P ✓
- **B** CLIQUE-10 $\in$ NP ✓
- **C** ~~CLIQUE-10 is NP-complete~~
- **D** ~~CLIQUE-10 $\notin$ NP~~
- **E** No idea. ✓

Algorithm

For all $v_1 \in V$:

For all $v_2 \in V_1$

⋮

For all $v_{10} \in V$:

$V' = \{v_1, \ldots, v_{10}\}$

Check if $G[V']$ is K_{10}

$\Theta(n^{10})$



→ *sli.do/advalg*

2.2 Models of Computation

Clicker Question



What is the cost of *adding* two (decimal) d-digit integers?
(For example, for $d = 5$, what is $45\,235 + 91\,342$?)

- **A** constant time
- **B** logarithmic in d
- **C** proportional to d
- **D** quadratic in d
- **E** no idea what you are talking about



→ *sli.do/advalg*

Clicker Question



What is the cost of *adding* two (decimal) d-digit integers?
(For example, for $d = 5$, what is $45\,235 + 91\,342$?)

- **A** constant time ✓ (uniform cost model)
- **B** ~~logarithmic in d~~
- **C** proportional to d ✓ (logarithmic cost model)
- **D** ~~quadratic in d~~
- **E** no idea what you are talking about ✓



→ *sli.do/advalg*

Mathematical Models of Computation

- complexity classes talk about sets of problems based upon whether they allow an algorithm of a certain cost
- in general, this depends on the allowable algorithms and their costs!

$\rightsquigarrow$ need to fix a machine model

Mathematical Models of Computation

- complexity classes talk about sets of problems based upon whether they allow an algorithm of a certain cost
- in general, this depends on the allowable algorithms and their costs!

$\rightsquigarrow$ need to fix a machine model

A *machine model* decides

- what algorithms are possible
- how they are described (= programming language)
- what an execution *costs*

Goal: Machine models should be
detailed and powerful enough to reflect actual machines,
abstract enough to unify architectures,
simple enough to analyze.

Random Access Machines

Standard model for detailed complexity analysis:

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures* by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- unlimited *memory* $\texttt{MEM}[0], \texttt{MEM}[1], \texttt{MEM}[2], \ldots$
- fixed number of *registers* $R_1, \ldots, R_r$ (say $r = 100$)

Random Access Machines

Standard model for detailed complexity analysis:

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures* by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- unlimited *memory* $\texttt{MEM}[0], \texttt{MEM}[1], \texttt{MEM}[2], \ldots$
- fixed number of *registers* $R_1, \ldots, R_r$ (say $r = 100$)
- memory cells $\texttt{MEM}[i]$ and registers R_i store w-bit integers, i. e., numbers in $[0..2^w - 1]$
  w is the word width/size; typically $\boxed{w \propto \lg n}$ $\rightsquigarrow$ $2^w \approx n$

Random Access Machines

Standard model for detailed complexity analysis:

Random access machine (RAM)

more detail in §2.2 of *Sequential and Parallel Algorithms and Data Structures* by Sanders, Mehlhorn, Dietzfelbinger, Dementiev

- unlimited *memory* $\texttt{MEM}[0], \texttt{MEM}[1], \texttt{MEM}[2], \ldots$
- fixed number of *registers* $R_1, \ldots, R_r$ (say $r = 100$)
- memory cells $\texttt{MEM}[i]$ and registers R_i store w-bit integers, i. e., numbers in $[0..2^w - 1]$
  w is the word width/size; typically $\boxed{w \propto \lg n} \rightsquigarrow 2^w \approx n$
- Instructions:
  - load & store: $R_i := \texttt{MEM}[R_j]$ $\texttt{MEM}[R_j] := R_i$
  - operations on registers: $R_k := R_i + R_j$ (arithmetic is *modulo* 2^w!)
    also $R_i - R_j$, $R_i \cdot R_j$, $R_i \operatorname{div} R_j$, $R_i \bmod R_j$
    C-style operations (bitwise and/or/xor, left/right shift)
  - conditional and unconditional jumps
- time cost: number of executed instructions
- space cost: total number of touched memory cells

RAM-Program Example

Example RAM program

```
1  // Assume: R_1 stores number N
2  // Assume: MEM[0..N) contains list of N numbers
3  R_2 := R_1;
4  R_3 := R_1 − 1;
5  R_5 := R_3 − 1;
6  R_4 := MEM[R_5];
7  R_6 := MEM[R_3];
8  if (R_4 ≤ R_6) goto line 11;
9  MEM[R_5] := R_6;
10 MEM[R_3] := R_4;
11 R_3 := R_3 − 1;
12 if (R_3 > 0) goto line 5;
13 R_2 := R_2 − 1;
14 if (R_2 > 0) goto line 4;
15 // Done:
```

RAM-Program Example

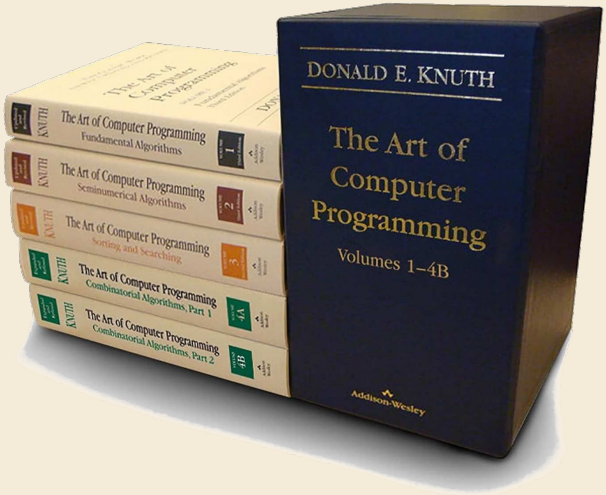
Example RAM program

```
1  // Assume: R_1 stores number N
2  // Assume: MEM[0..N) contains list of N numbers
3  R_2 := R_1;
4  R_3 := R_1 − 1;
5  R_5 := R_3 − 1;
6  R_4 := MEM[R_5];
7  R_6 := MEM[R_3];
8  if (R_4 ≤ R_6) goto line 11;
9  MEM[R_5] := R_6;
10 MEM[R_3] := R_4;
11 R_3 := R_3 − 1;
12 if (R_3 > 0) goto line 5;
13 R_2 := R_2 − 1;
14 if (R_2 > 0) goto line 4;
15 // Done: MEM[0..N) sorted
```

RAM-Program Example

Example RAM program

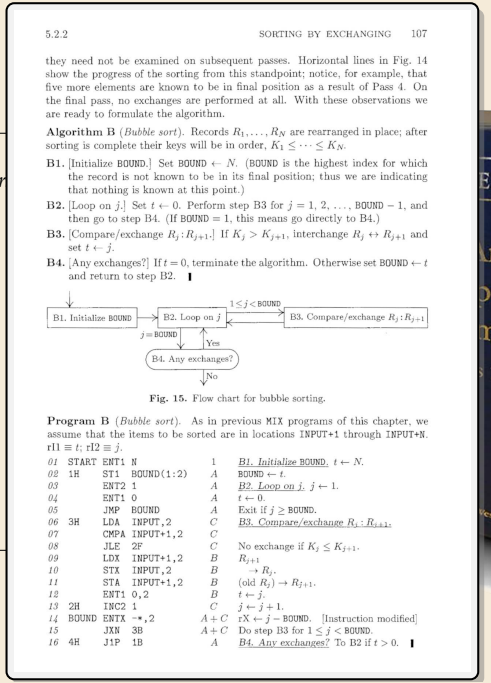
```
1  // Assume: R_1 stores number N
2  // Assume: MEM[0..N) contains list of N numbers
3  R_2 := R_1;
4  R_3 := R_1 − 1;
5  R_5 := R_3 − 1;
6  R_4 := MEM[R_5];
7  R_6 := MEM[R_3];
8  if (R_4 ≤ R_6) goto line 11;
9  MEM[R_5] := R_6;
10 MEM[R_3] := R_4;
11 R_3 := R_3 − 1;
12 if (R_3 > 0) goto line 5;
13 R_2 := R_2 − 1;
14 if (R_2 > 0) goto line 4;
15 // Done: MEM[0..N) sorted
```



RAM-Program Example

Example RAM program

1 // *Assume:* R_1 *stores number* N
2 // *Assume:* MEM[0..N] *contains list of* N *numbers*
3 $R_2 := R_1$;
4 $R_3 := R_1 - 1$;
5 $R_5 := R_3 - 1$;
6 $R_4 := \mathtt{MEM}[R_5]$;
7 $R_6 := \mathtt{MEM}[R_3]$;
8 **if** $(R_4 \leq R_6)$ goto line 11;
9 $\mathtt{MEM}[R_5] := R_6$;
10 $\mathtt{MEM}[R_3] := R_4$;
11 $R_3 := R_3 - 1$;
12 **if** $(R_3 > 0)$ goto line 5;
13 $R_2 := R_2 - 1$;
14 **if** $(R_2 > 0)$ goto line 4;
15 // *Done:* MEM[0..N] *sorted*



5.2.2 SORTING BY EXCHANGING 107

they need not be examined on subsequent passes. Horizontal lines in Fig. 14 show the progress of the sorting from this standpoint; notice, for example, that five more elements are known to be in final position as a result of Pass 4. On the final pass, no exchanges are performed at all. With these observations we are ready to formulate the algorithm.

Algorithm B (*Bubble sort*). Records $R_1, \ldots, R_N$ are rearranged in place; after sorting is complete their keys will be in order, $K_1 \leq \cdots \leq K_N$.

B1. [Initialize BOUND.] Set BOUND $\leftarrow N$. (BOUND is the highest index for which the record is not known to be in its final position; thus we are indicating that nothing is known at this point.)

B2. [Loop on j.] Set $t \leftarrow 0$. Perform step B3 for $j = 1, 2, \ldots,$ BOUND $- 1$, and then go to step B4. (If BOUND $= 1$, this means go directly to B4.)

B3. [Compare/exchange $R_j : R_{j+1}$.] If $K_j > K_{j+1}$, interchange $R_j \leftrightarrow R_{j+1}$ and set $t \leftarrow j$.

B4. [Any exchanges?] If $t = 0$, terminate the algorithm. Otherwise set BOUND $\leftarrow t$ and return to step B2. ∎

Fig. 15. Flow chart for bubble sorting.

Program B (*Bubble sort*). As in previous MIX programs of this chapter, we assume that the items to be sorted are in locations INPUT+1 through INPUT+N. r11 $\equiv t$; r12 $\equiv j$.

```
01  START ENT1 N          1     B1. Initialize BOUND. t ← N.
02  1H    ST1  BOUND(1:2) A     BOUND ← t.
03        ENT2 1          A     B2. Loop on j. j ← 1.
04        ENT1 0          A     t ← 0.
05        JMP  BOUND      A     Exit if j ≥ BOUND.
06  3H    LDA  INPUT,2    C     B3. Compare/exchange R_j : R_{j+1}:
07        CMPA INPUT+1,2  C
08        JLE  2F         C     No exchange if K_j ≤ K_{j+1}.
09        LDX  INPUT+1,2  B     R_{j+1}
10        STX  INPUT,2    B       → R_j.
11        STA  INPUT+1,2  B     (old R_j) → R_{j+1}.
12        ENT1 0,2        B     t ← j.
13  2H    INC2 1          C     j ← j + 1.
14  BOUND ENTX *-*,2      A+C   rX ← j − BOUND.  [Instruction modified]
15        JXN  3B         A+C   Do step B3 for 1 ≤ j < BOUND.
16  4H    J1P  1B         A     B4. Any exchanges? To B2 if t > 0. ∎
```

2.3 Turing Machines

Keep it Simple, Stupid

- word-RAM (rather) realistic, but complicated
  - note that the machine has to grow with the inputs(!)
- for a coarse distinction of running time complexity, simpler models suffice
  - useful to reason about “all algorithms”
  - machine is fixed for all inputs sizes apart from storage for input

Keep it Simple, Stupid

- word-RAM (rather) realistic, but complicated
  - note that the machine has to grow with the inputs(!)
- for a coarse distinction of running time complexity, simpler models suffice
  - useful to reason about "all algorithms"
  - machine is fixed for all inputs sizes apart from storage for input

Many models of computation . . .

- μ-recursive function
- Turing machines (TM)
- counter machines
- λ-calculus
- While-programs
- any Turing-complete language
- quantum computers
- . . .

. . . with strong equivalences:

1. all proven to lead to the *same* set of computable functions
2. **Church-Turing thesis**:
   any formalization of "effectively computable" is equivalent in this sense
3. ***Extended Church-Turing Thesis:***
   . . . and can be simulated with *polynomial overhead* on a TM
   - true for all on left . . .
   - except theoretical quantum computers!
   - ignore them for now ← wake me when they exist

Turing Machines

- invented by ***Alan Turing*** in 1936 as formalization for "computable by hand"

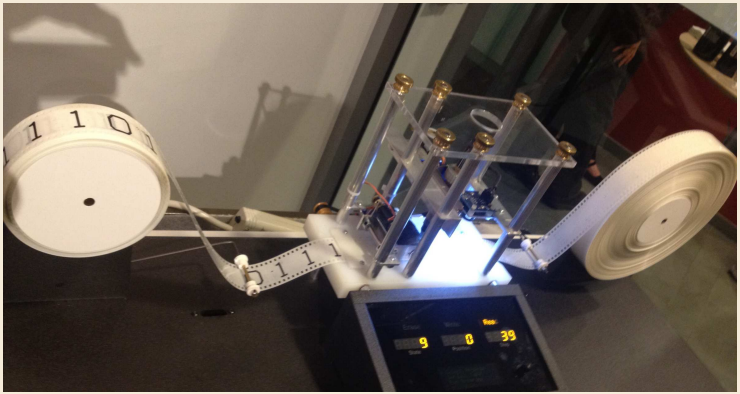
  In same paper, Turing proved undecidability of halting problem!

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. Turing.

[Received 28 May, 1936.—Read 12 November, 1936.]

- *minimalistic* model of universal computer, but can be built:



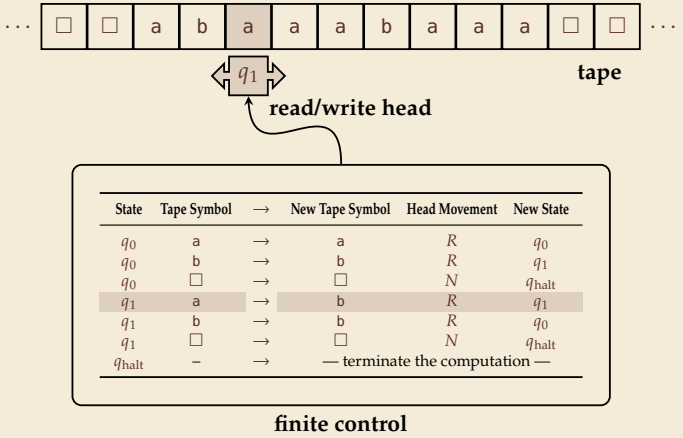
Turing Machines in Circulation



Turing Machines – Informal Recap

A *Turing machine* has

- a **finite control** via states
- an **input/output-tape**
  - unbounded length
  - initially contains input
  - all other cells contain "□"
- a **read/write head**
  - reads the current symbol
  - overwrites it with a new symbol
  - initially placed on beginning of input



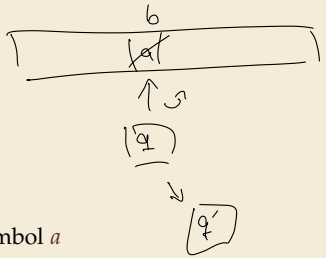
Turing Machines – Formal Syntax

Definition 2.1 (Turing Machine (TM))

A *Turing machine* is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, q_{\text{halt}})$ with

- a finite set of *states* Q,
- an *input alphabet* Σ,
- a *tape alphabet* $\Gamma \supset \Sigma$,
- for deterministic TMs a *transition function* $\delta : (Q \setminus \{q_{\text{halt}}\}) \times \Gamma \to Q \times \Gamma \times \{L, R, N\}$
  for **non**deterministic TMs a *transition relation* $\delta : (Q \setminus \{q_{\text{halt}}\}) \times \Gamma \to 2^{Q \times \Gamma \times \{L,R,N\}}$
- an *initial state* $q_0 \in Q$,
- a *blank symbol* $\square \in \Gamma \setminus \Sigma$, and
- a halting state $q_{\text{halt}} \in Q$

Turing Machine – Computation Step



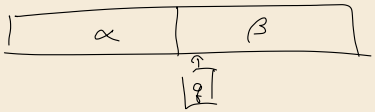
- Each step of a computation of TM M has the form
  $\delta(q, a) = (q', b, d)$ resp. $\delta(q, a) \ni (q', b, d)$,
  with the semantics that
  - M is in state $q \neq q_{\text{halt}}$
  - the cell below the read/write head currently contains symbol a
  - M now changes (based on its finite control)
    - into state q',
    - writes b into the cell under the read/write head
    - and finally moves the read/write head in direction $d \in \{L, R, N\}$.
      (L = left, R = right, N = none (stay))
- for deterministic TM M, q and a uniquely determine this action;
  for nondeterministic TM, we may have several possible actions.
- to formally define an entire computation, we have to encode the tape contents as well

Turing Machines – Configurations

Definition 2.2 (TM Configuration)

A *configuration (config)* of a TM M is a string $C \in \Gamma^\star Q\, \Gamma^+$.
We identify configs differing in *leading* $\square$s of α or *trailing* $\square$s of β (keeping $\beta \neq \varepsilon$) and use the shortest representative. ◀

The semantics of config $C = \alpha q \beta$, $q \in Q$, is: tape content $\alpha\beta$ (all other cells blank) and head at first symbol of β.



Turing Machines – Configurations

Definition 2.2 (TM Configuration)

A *configuration (config)* of a TM M is a string $C \in \Gamma^\star Q\,\Gamma^+$.

We identify configs differing in *leading* $\square$s of α or *trailing* $\square$s of β (keeping $\beta \neq \varepsilon$) and use the shortest representative. ◀

The semantics of config $C = \alpha q \beta$, $q \in Q$, is: tape content $\alpha\beta$ (all other cells blank) and head at first symbol of β.

Definition 2.3 (TM Computation Relation)

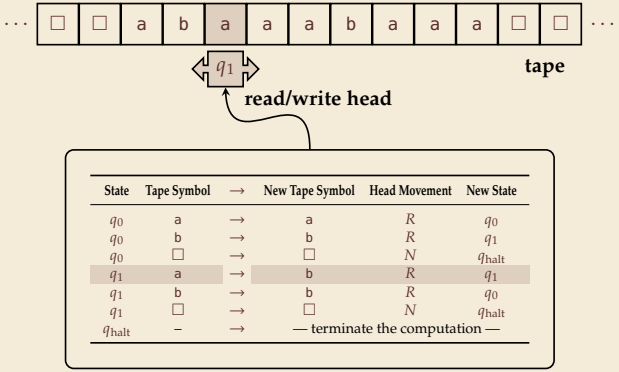
The *computation relation* $\vdash$ is defined on the set of configurations of a TM M as follows.

$$a_1 \ldots a_m\, q\, b_1 \ldots b_n \vdash \begin{cases} a_1 \ldots a_m \;\; q'\, c\, b_2 \ldots b_n, & \delta(q, b_1) \ni (q', c, N) \\ a_1 \ldots a_m\, c \;\; q'\, b_2 \ldots b_n, & \delta(q, b_1) \ni (q', c, R),\ n \geq 2, \\ a_1 \ldots a_{m-1}\, q'\, a_m\, c\, b_2 \ldots b_n, & \delta(q, b_1) \ni (q', c, L),\ m \geq 1. \end{cases}$$

(For $n = 1$ resp. $m = 0$, apply the rule to $\alpha\, q\, \beta\square$ resp. $\square\alpha\, q\, \beta$.) ◀

Turing Machines – Configuration Example

Example:
For the shown TM M,
the current configuration is:
$C = \mathtt{ab}\, q_1\, \mathtt{aaabaaa}$



State	Tape Symbol	→	New Tape Symbol	Head Movement	New State
q_0	a	→	a	R	q_0
q_0	b	→	b	R	q_1
q_0	□	→	□	N	q_{halt}
q_1	a	→	b	R	q_1
q_1	b	→	b	R	q_0
q_1	□	→	□	N	q_{halt}
q_{halt}	–	→	— terminate the computation —		

- TM Config $C = \alpha q \beta$ completely describes current state of computation
  - $\alpha\beta$ is the (non-blank) tape content
  - q is the current state of the TM
  - the read/write head is on the first symbol of β

Turing Machines – Computed Function

- With this setup, we can now formally define what a Turing machine computes.

Definition 2.4 (Function computed by a TM)

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, q_{\text{halt}})$ be a TM. The function computed by M on input $x \in \Sigma^\star$, written $M(x)$, is defined as
$M(x) = \{y : q_0 x \vdash^\star q_{\text{halt}}\, y\}$.
For deterministic TMs, we will also have $|M(x)| \leq 1$ and we write $M(x) = y$ for $M(x) = \{y\}$.

Turing Machines – Computed Function

- With this setup, we can now formally define what a Turing machine computes.

Definition 2.4 (Function computed by a TM)

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, q_{\text{halt}})$ be a TM. The function computed by M on input $x \in \Sigma^\star$, written $M(x)$, is defined as
$M(x) = \{y : q_0 x \vdash^\star q_{\text{halt}}\, y\}$.
For deterministic TMs, we will also have $|M(x)| \leq 1$ and we write $M(x) = y$ for $M(x) = \{y\}$. ◀

Definition 2.5 (Time and Space cost)

For a TM M and input $x \in \Sigma^\star$, we define
$time_M(x) = \sup\{t : q_0 x \vdash^t q_{\text{halt}} y\} \cup \{0\}$. worst case over all computations
We define $space_M(x) = \sup\{|\alpha\beta| : q_0 x \vdash^\star \alpha q \beta\} \cup \{0\}$. ◀

- Note: *time* and *space* can be ∞ or 0 for nondeterministic TMs.

Turing Machines – Accepted Language

- Often convenient to use language acceptance instead of function computation.
- for deterministic TM, compute *characteristic function* of L: $\mathbb{1}_L(x) = \begin{cases} 1 & \text{if } x \in L \\ 0 & \text{otherwise} \end{cases}$
- care needed for nondeterministic TM

Turing Machines – Accepted Language

- Often convenient to use language acceptance instead of function computation.
- for deterministic TM, compute *characteristic function* of L: $\mathbb{1}_L(x) = \begin{cases} 1 & \text{if } x \in L \\ 0 & \text{otherwise} \end{cases}$
- care needed for nondeterministic TM

Definition 2.6 (Language of TM)

The language $\mathcal{L}(M)$ accepted by a TM M is defined as

$$\mathcal{L}(M) \quad = \quad \{w \in \Sigma^\star : 1 \in M(w)\}.$$

0, 1 ∈ Γ

$\rightsquigarrow$ nondeterministic TM accepts w iff some computation accepts w

Turing Machines – Several tapes

Remark 2.7 (k-tape TMs)

We only consider one-tape TMs here. In general, k-tape TMs can be faster.
However, any language accepted by a k-tape TM in time $f(n)$
is also accepted by a 1-tape TM with running time $O(f^2(n))$.
The models are thus *polynomially equivalent*.

Turing Machines – Totality

- In complexity theory, we will restrict ourselves to TMs that always halt.

Definition 2.8 (Terminating TM)

A TM M is *always terminating / total* if there is a function $T : \mathbb{N} \to \mathbb{N}$ such that $q_0 x = C_0 \vdash C_1 \vdash \cdots \vdash C_t$ implies $t \leq T(|x|)$, and there is a y such that $q_0 x \vdash^\star q_{\text{halt}} y$. ◀

- Note that in a terminating TM, we always have $1 \leq time_M(x) \leq T(|x|)$.

Turing Machines – Totality

- In complexity theory, we will restrict ourselves to TMs that always halt.

Definition 2.8 (Terminating TM)

A TM M is *always terminating / total* if there is a function $T : \mathbb{N} \to \mathbb{N}$ such that $q_0 x = C_0 \vdash C_1 \vdash \cdots \vdash C_t$ implies $t \leq T(|x|)$, and there is a y such that $q_0 x \vdash^\star q_{\text{halt}} y$. ◀

- Note that in a terminating TM, we always have $1 \leq time_M(x) \leq T(|x|)$.

Lemma 2.9 (Time-constrained TM)

Given a (potentially nonterminating) TM M and a function $T : \mathbb{N} \to \mathbb{N}$ **computable** in time $T(n)$, we can construct an *always terminating* TM M' that simulates M on x for $T(|x|)$ steps and outputs `TIMEOUT` if M has not terminated yet. Moreover, $time_{M'}(x) = O(T^2(|x|))$ for all $x \in \Sigma^\star$. ◀

Turing Machines – Totality

- In complexity theory, we will restrict ourselves to TMs that always halt.

Definition 2.8 (Terminating TM)

A TM M is *always terminating / total* if there is a function $T : \mathbb{N} \to \mathbb{N}$ such that $q_0 x = C_0 \vdash C_1 \vdash \cdots \vdash C_t$ implies $t \leq T(|x|)$, and there is a y such that $q_0 x \vdash^\star q_{\text{halt}} y$. ◀

- Note that in a terminating TM, we always have $1 \leq time_M(x) \leq T(|x|)$.

Lemma 2.9 (Time-constrained TM)

Given a (potentially nonterminating) TM M and a function $T : \mathbb{N} \to \mathbb{N}$ **computable** in time $T(n)$, we can construct an *always terminating* TM M' that simulates M on x for $T(|x|)$ steps and outputs `TIMEOUT` if M has not terminated yet. Moreover, $time_{M'}(x) = O(T^2(|x|))$ for all $x \in \Sigma^\star$. ◀

$\rightsquigarrow$ If we are only interested in the (non)existence of a polynomial-time TM, we can restrict ourselves to total TMs that will never `TIMEOUT`.

Models of Computation – Summary

- Concrete model such as TMs useful for some proofs
- Often, details do not matter as long as models are polynomially equivalent
  - Note: TM always means we are in the logarithmic cost model for arithmetic operations
- In the following, discuss more abstract notion of "algorithm"
  (fine to substitute by TM in each case)

2.4 The Classes P and NP

Worst Case Complexity

Definition 2.10 (Time and Space Complexity – Generic)

Let Σ_I and Σ_O two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^\star \to \Sigma_O^\star$. Then for each $x \in \Sigma_I^\star$ we denote by $time_A(x)$ (resp. $space_A(x)$) the logarithmic time complexity (resp. logarithmic space complexity) for A on x. ◀

Worst Case Complexity

Definition 2.10 (Time and Space Complexity – Generic)

Let Σ_I and Σ_O two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^\star \to \Sigma_O^\star$. Then for each $x \in \Sigma_I^\star$ we denote by $time_A(x)$ (resp. $space_A(x)$) the logarithmic time complexity (resp. logarithmic space complexity) for A on x. ◀

Where needed, we can unpack this in full detail for Turing machines!

Worst Case Complexity

Definition 2.10 (Time and Space Complexity – Generic)

Let Σ_I and Σ_O two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^\star \to \Sigma_O^\star$. Then for each $x \in \Sigma_I^\star$ we denote by $time_A(x)$ (resp. $space_A(x)$) the logarithmic time complexity (resp. logarithmic space complexity) for A on x. ◀

Where needed, we can unpack this in full detail for Turing machines!

Definition 2.11 (Worst-Case Complexity)

Let Σ_I and Σ_O be two alphabets and A an algorithm implementing a **total** mapping $\Sigma_I^\star \to \Sigma_O^\star$. The *worst case time complexity of* A is the function $Time_A : \mathbb{N} \to \mathbb{N}$ with

$$Time_A(n) \quad = \quad \max\{time_A(x) : x \in \Sigma_I^{\leq n}\},$$

for each $n \in \mathbb{N}$. The *worst case space complexity of* A is given by function $Space_A : \mathbb{N} \to \mathbb{N}$ with

$$Space_A(n) \quad = \quad \max\{space_A(x) : x \in \Sigma_I^n\}.$$

◀

Decision Problems = Languages

Definition 2.12 (Decision Problem and Algorithms)

A *decision problem* is given by $P = (L, U, \Sigma)$ for Σ an alphabet and $L \subseteq U \subseteq \Sigma^\star$. An algorithm A *solves (decides)* decision problem P, if for all $x \in U$

1. $A(x) = 1$ for $x \in L$, and
2. $A(x) = 0$ for $x \in U \setminus L$ (i. e., $x \notin L$)

holds. Here $A(x)$ denotes the output of A on input x.
If $U = \Sigma^\star$ holds we denote P briefly by (L, Σ). ◀

$\rightsquigarrow$ A computes a total function $A : U \to \{0, 1\}$,
the *characteristic function* $\mathbb{1}_L : U \to \{0, 1\}$ of language L.
We then write $L = \mathcal{L}(A)$, the language accepted by A.

Decision Problems = Languages

Definition 2.12 (Decision Problem and Algorithms)

A *decision problem* is given by $P = (L, U, \Sigma)$ for Σ an alphabet and $L \subseteq U \subseteq \Sigma^\star$. An algorithm A *solves (decides)* decision problem P, if for all $x \in U$

1. $A(x) = 1$ for $x \in L$, and
2. $A(x) = 0$ for $x \in U \setminus L$ (i. e., $x \notin L$)

holds. Here $A(x)$ denotes the output of A on input x.
If $U = \Sigma^\star$ holds we denote P briefly by (L, Σ).

$\rightsquigarrow$ A computes a total function $A : U \to \{0, 1\}$,
the *characteristic function* $\mathbb{1}_L : U \to \{0, 1\}$ of language L.
We then write $L = \mathcal{L}(A)$, the language accepted by A.

We restrict our attention to ***decision problems***:
Given: $w \in \Sigma^\star$.
Goal: Is $w \in L$?

Example: "sensible encoding"

w is an encoding of an instance of the traveling salesperson problem **and** a threshold D

$L = \{w : w \text{ encodes instance w/ opt. round trip length} \leq D\}$

Optimal Algorithms

Definition 2.13 (Upper/Lower Bounds, Optimal Algorithms)

Let U be an algorithmic problem and f, g functions $\mathbb{N}_0 \to \mathbb{R}^+$.

- We call $O(g(n))$ an *upper bound for time complexity of* U if there is an algorithm A that solves U in time $Time_A(n) \in O(g(n))$.
- We say $\Omega(f(n))$ is a *lower bound for time complexity of* U if every algorithm A that solves U needs time $Time_A(n) \in \Omega(f(n))$.
- An algorithm A is called "optimal" for U if $Time_A(n) \in O(g(n))$ and $\Omega(g(n))$ is a lower bound for the time complexity of U.

Running Time

Definition 2.14 (time classes)

For a function $f : \mathbb{N} \to \mathbb{N}$, the class $TIME(f(n))$ is the set of all languages A for which there is a *deterministic*, *always terminating* Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ for all $w \in \Sigma^\star$. ◀

- (For **non-terminating** M, $time_M(w) = 0$ would make the bound vacuous;
  by the Time-constrained TM lemma, requiring termination costs nothing for polynomial bounds.)

Running Time

Definition 2.14 (time classes)

For a function $f : \mathbb{N} \to \mathbb{N}$, the class $TIME(f(n))$ is the set of all languages A for which there is a *deterministic*, *always terminating* Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ for all $w \in \Sigma^\star$. ◀

- (For **non-terminating** M, $time_M(w) = 0$ would make the bound vacuous; by the Time-constrained TM lemma, requiring termination costs nothing for polynomial bounds.)

Definition 2.15 (P, tractable)

We define the class of languages P decidable in polynomial time by

$$\mathsf{P} := \bigcup_{p \text{ polynomial}} TIME(p(n)).$$

$TIME(n^{O(1)})$

A language (a decision problem) $L \in \mathsf{P}$ is called *tractable / efficiently decidable*. ◀

Nondeterministic Running Time

Recall:

- A nondeterministic Turing machine / algorithm M accepts L ($\mathcal{L}(M) = L$) if for all $x \in L$ there is at least one computation of M which accepts x and for all $y \notin L$ no computation of M accepts y.
- We only consider always-terminating Turing machines.
- The running time $time_M(x)$ of M on x is given by the longest computation of M on x.

Nondeterministic Running Time

Recall:

- A nondeterministic Turing machine / algorithm M accepts L ($\mathcal{L}(M) = L$) if for all $x \in L$ there is at least one computation of M which accepts x and for all $y \notin L$ no computation of M accepts y.
- We only consider always-terminating Turing machines.
- The running time $time_M(x)$ of M on x is given by the longest computation of M on x.

Definition 2.16 (NTIME, NP)

For function $f : \mathbb{N} \to \mathbb{N}$, the class $NTIME(f(n))$ is the set of all languages A, for which there is a ***non****deterministic* Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ for all $w \in \Sigma^\star$.

Nondeterministic Running Time

Recall:

- A nondeterministic Turing machine / algorithm M accepts L ($\mathcal{L}(M) = L$) if for all $x \in L$ there is at least one computation of M which accepts x and for all $y \notin L$ no computation of M accepts y.
- We only consider always-terminating Turing machines.
- The running time $time_M(x)$ of M on x is given by the longest computation of M on x.

Definition 2.16 (NTIME, NP)

For function $f : \mathbb{N} \to \mathbb{N}$, the class $NTIME(f(n))$ is the set of all languages A, for which there is a ***non**deterministic* Turing machine M with $\mathcal{L}(M) = A$ and $time_M(w) \leq f(|w|)$ for all $w \in \Sigma^\star$. The class of languages NP is defined by

$$\mathsf{NP} := \bigcup_{p \text{ polynomial}} NTIME(p(n)).$$


$true_A(k)$

2.5 Nondeterminism = Verification

Nondeterminism?

- The original definition of NP via nondeterministic Turing machines is not very intuitive.
- There is an equivalent characterization that is usually more convenient to use: Certificates and verifiers.

Polynomially verifiable

Definition 2.17 (Certificates, Verifier, VP)

Let $L \subseteq \Sigma^\star$ be a language.

- An algorithm A acting on inputs from $\Sigma^\star \times \{0,1\}^\star$ is called *verifier for* L (notation $L = \mathcal{V}(A)$), if

$$L = \{w \in \Sigma^\star : \exists c \in \{0,1\}^\star \; A(w,c) = 1\}.$$

If A accepts input (w,c) we say c is *proof* or *certificate* for $w \in L$.

If $w \in L$ then $\exists$ certificate c $A(w,c)$ ✓

If $w \notin L$ $\forall$ c $A(w,c)$ ✗

Polynomially verifiable

Definition 2.17 (Certificates, Verifier, VP)

Let $L \subseteq \Sigma^\star$ be a language.

- An algorithm A acting on inputs from $\Sigma^\star \times \{0,1\}^\star$ is called *verifier for* L (notation $L = \mathcal{V}(A)$), if
$$L \;=\; \{w \in \Sigma^\star : \exists c \in \{0,1\}^\star \; A(w,c) = 1\}.$$
If A accepts input (w,c) we say c is *proof* or *certificate* for $w \in L$.

- A verifier A for L is a *polynomial-time verifier* if there is a polynomial p such that $time_A(w,c) \leq p(|w|)$ **for all** $w \in \Sigma^\star$ and $c \in \{0,1\}^\star$.
Note: A can then read at most $p(|w|)$ bits of c, so w.l.o.g. $|c| \leq p(|w|)$.
$\rightsquigarrow$ Certificates are short.

Polynomially verifiable

Definition 2.17 (Certificates, Verifier, VP)

Let $L \subseteq \Sigma^\star$ be a language.

- An algorithm A acting on inputs from $\Sigma^\star \times \{0,1\}^\star$ is called *verifier for* L (notation $L = \mathcal{V}(A)$), if
$$L = \{w \in \Sigma^\star : \exists c \in \{0,1\}^\star \; A(w,c) = 1\}.$$
If A accepts input (w,c) we say c is *proof* or *certificate* for $w \in L$.

- A verifier A for L is a *polynomial-time verifier* if there is a polynomial p such that $time_A(w,c) \leq p(|w|)$ **for all** $w \in \Sigma^\star$ and $c \in \{0,1\}^\star$.
Note: A can then read at most $p(|w|)$ bits of c, so w.l.o.g. $|c| \leq p(|w|)$.
$\rightsquigarrow$ Certificates are short.

- We define the class of polynomially verifiable languages VP by
$$\mathsf{VP} = \{\mathcal{V}(A) : A \text{ is polynomial time verifier}\}.$$ ◀

Decision version of TSP $\in$ VP ✓ w encodes cities, distances D, use c to select edges i,j $\Rightarrow$ c encodes one tour

Nondeterminism ↔ certificate

Theorem 2.18

NP = VP.

Proof: